

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Пишем сканер портов

Сканер портов - это программа которая ищет и перечисляет открытые на хосте порты. Это могут быть как определённые порты, так и диапазон портов. Соответствующие сканеры используются как администраторами/пентестерами для проверки безопасности сетей, так и злоумышленниками.

Назначением портов занимается **IANA** (**Internet Assigned Numbers Authority** — «Администрация адресного пространства Интернет») она выполняет функцию управления пространствами IP-адресов, доменов верхнего уровня, а также регистрирующая типы данных MIME, и параметры прочих протоколов Интернета.

Диапазоны портов разбиты на несколько частей:

Часть портов относится к так называемым **системным или общеизвестным**. Они имеют номера от **0 до 1023**. В этом диапазоне находятся зарегистрированные номера портов для определённых служб.

Примеры общеизвестных портов:

21 - FTP (File Transfer Protocol) Протокол передачи файлов. FTP позволяет подключаться к серверам, просматривать содержимое каталогов, и загружать файлы с сервера или на сервер.

22 - SSH (Secure Secure Shell) Сетевой протокол, позволяющий производить удалённое управление операционной системой используя шифрование.

23 - TELNET (TELecommunication NETwork) Позволяет управлять Операционной Системой удаленно.

25 - STMP (Simple Mail Transfer Protocol) простой протокол передачи почты.

53 - DNS (Domain Name System) Система показывающая информацию о доменах.

80 - HTTP (HyperText Transfer Protocol) протокол передачи гипертекста.

443 - HTTPS (HyperText Transfer Protocol Secure) протокол защищённой передачи гипертекста.

Зарегистрированные порты имеют номера от **1024 до 49151**.

Назначаются **IANA** для конкретной услуги по ходатайству запрашивающего лица. В большинстве систем, зарегистрированные порты могут быть использованы без привилегий суперпользователя.

Частотные или динамические порты находятся в диапазоне от **29152 до 65535**.

Одним из вариантов использования данного диапазона являются временные порты.

Чтобы просканировать на открытые порты определённый ресурс, напомним простенький код, в котором мы с помощью цикла прогоним запросы к портам диапазона 0-443. Если мы хотим проверить все возможные варианты, то необходимо сканировать по максимально возможное значение 0-65535.

Значение таймаута нужно подбирать в зависимости от скорости ответа сервера. Если сервер не успеет ответить, то открытый порт будет пропущен. Логика работы программы простая – если подключение к серверу удалось, значит порт открыт.

```
import socket

def port_scan(hostname, port_num):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(1)
    try:
        s.connect((hostname, port_num))
        print(hostname, 'Port:', port_num, "is open")
    except socket.error:
        pass
    finally:
        s.close()

ip = '52.89.18.56'
print('Start scan...')
for i in range(444):
    port_scan(ip, i)
print('Scan completed!')
```

В результате сканирования, обнаружено 2 открытых порта.

```
one_host x
Start scan...
52.89.18.56 Port: 80 is open
52.89.18.56 Port: 443 is open
Scan completed!

Process finished with exit code 0
```

Чтобы просканировать диапазон IP-адресов, применим модуль `netaddr`. Функция `IPRange` принимает 2 параметра – начало и конец диапазона IP-адресов. Ввод пользователю лучше всего сделать через дефис, разделяя начало и конец диапазона через `split`.

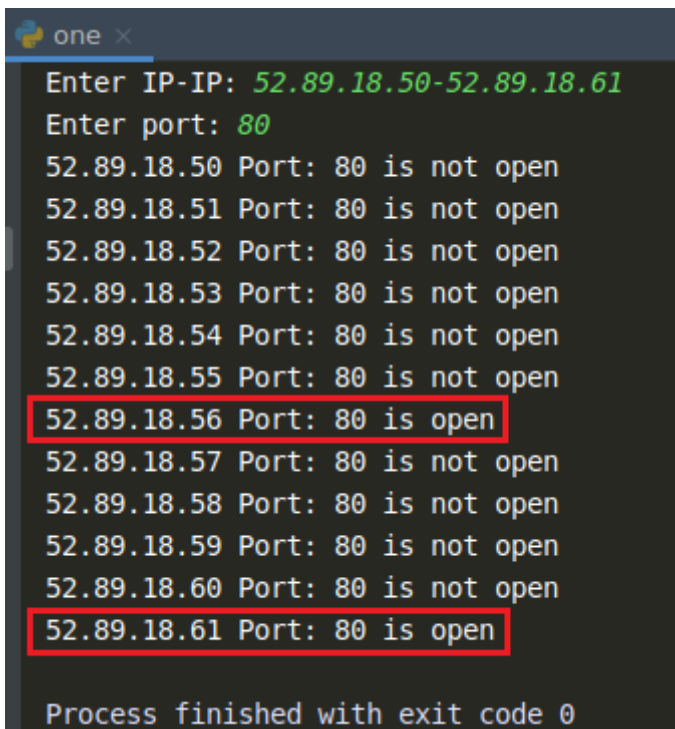
```

import socket
from netaddr import IPRange

def port_scan(hostname, port_num):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)
    try:
        s.connect((hostname, port_num))
        print(hostname, 'Port:', port_num, "is open")
    except socket.error:
        print(hostname, 'Port:', port_num, "is not open")
    finally:
        s.close()

ipStart, ipEnd = input("Enter IP-IP: ").split("-")
iprange = IPRange(ipStart, ipEnd)
port = int(input('Enter port: '))
for ip in iprange:
    host = str(ip)
    port_scan(host, port)

```



```

one x
Enter IP-IP: 52.89.18.50-52.89.18.61
Enter port: 80
52.89.18.50 Port: 80 is not open
52.89.18.51 Port: 80 is not open
52.89.18.52 Port: 80 is not open
52.89.18.53 Port: 80 is not open
52.89.18.54 Port: 80 is not open
52.89.18.55 Port: 80 is not open
52.89.18.56 Port: 80 is open
52.89.18.57 Port: 80 is not open
52.89.18.58 Port: 80 is not open
52.89.18.59 Port: 80 is not open
52.89.18.60 Port: 80 is not open
52.89.18.61 Port: 80 is open
Process finished with exit code 0

```

Чтобы открытый порт выделялся, можно его покрасить в зелёный цвет. Однако если количество адресов большое, гораздо разумнее выводить только открытые порты, поставив в эксепшене заглушку `pass`.

Традиционной формой записи IPv4 адреса является запись в виде четырёх десятичных чисел (от 0 до 255), разделённых точками.

Довольно часто применяются записи вида **52.89.18.0/24**. Через дробь указывается длина маски подсети. Данный вид записи заменяет собой указание диапазона IP-адресов, и будет соответствовать **52.89.18.0-52.89.18.255**. Подробнее можно посмотреть на [этом ресурсе](#).

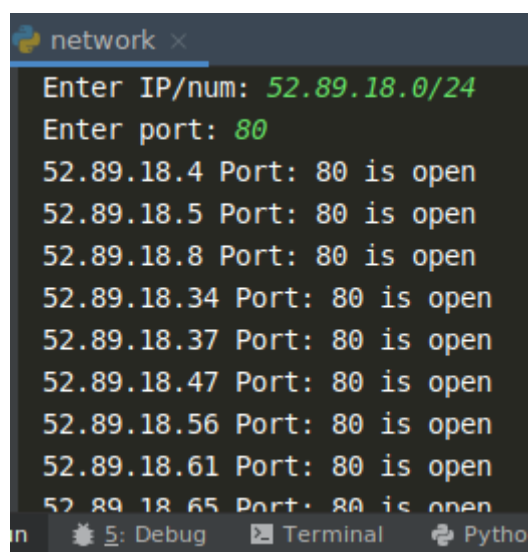
Функция **IPNetwork** работает с маской подсети, соответствующий код будет выглядеть так:

```
import socket
from netaddr import IPNetwork

def port_scan(hostname, port_num):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)
    try:
        s.connect((hostname, port_num))
        print(hostname, 'Port:', port_num, "is open")
    except socket.error:
        pass
    finally:
        s.close()

ip_scale = input("Enter IP/num: ")
iprange = IPNetwork(ip_scale)
port = int(input('Enter port: '))
for ip in iprange:
    host = str(ip)
    port_scan(host, port)

# 52.89.18.0/24
```



```
network x
Enter IP/num: 52.89.18.0/24
Enter port: 80
52.89.18.4 Port: 80 is open
52.89.18.5 Port: 80 is open
52.89.18.8 Port: 80 is open
52.89.18.34 Port: 80 is open
52.89.18.37 Port: 80 is open
52.89.18.47 Port: 80 is open
52.89.18.56 Port: 80 is open
52.89.18.61 Port: 80 is open
52.89.18.65 Port: 80 is open
n 5: Debug Terminal Python
```

Однопоточные сканеры конечно выполняют работу очень медленно, поэтому добавим скорости применив многопоточность.

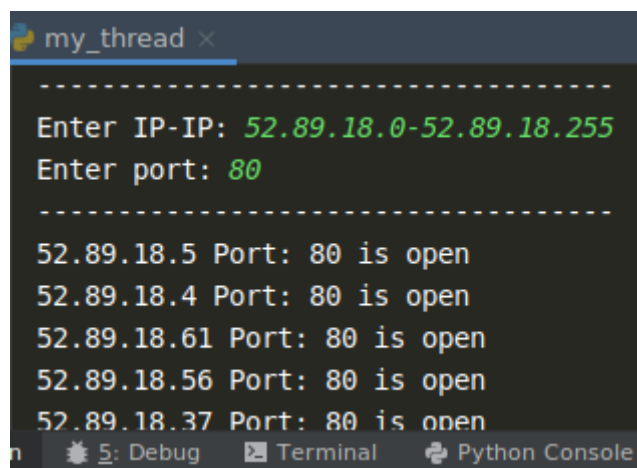
```
import threading
import socket
from netaddr import IPRange

def port_scan(hostname, port_num):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)
    try:
        s.connect((hostname, port_num))
        print(hostname, 'Port:', port_num, "is open")
    except socket.error:
        pass
    finally:
        s.close()

print('-' * 36)
ipStart, ipEnd = input("Enter IP-IP: ").split("-")
iprange = IPRange(ipStart, ipEnd)
port = int(input('Enter port: '))
print('-' * 36)

if __name__ == '__main__':
    for ip in iprange:
        host = str(ip)
        t = threading.Thread(target=port_scan, args=(host, port))
        t.start()

# 52.89.18.0-52.89.18.255
```



```
my_thread x
-----
Enter IP-IP: 52.89.18.0-52.89.18.255
Enter port: 80
-----
52.89.18.5 Port: 80 is open
52.89.18.4 Port: 80 is open
52.89.18.61 Port: 80 is open
52.89.18.56 Port: 80 is open
52.89.18.37 Port: 80 is open
n 5: Debug Terminal Python Console
```